

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA PRÁCTICA – Práctica 1

Presentación

En la PEC 1 se han analizado diferentes estilos arquitectónicos y los criterios para seleccionar cuál o cuáles de ellos son los más adecuados para guiar el desarrollo de una solución.

En esta Práctica 1 veremos cómo diseñar una solución basada en una arquitectura de microservicios, usando modelos de representación y patrones de diseño.

La Práctica 1 alcanza los contenidos del libro "Microservices Patterns" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En la Práctica 1 se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Capacidad de analizar un problema en el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y resolverlo.

Objetivos

Los objetivos de la Práctica 1 son:

- Identificar operaciones que un sistema tiene que proporcionar.
- Proponer el diseño de una arquitectura y su descomposición en microservicios.
- Diseñar de forma detallada un microservicio.
- Evaluar y justificar cómo resolver una problemática concreta en el diseño de microservicios (escoger un patrón, un estilo de comunicación o resolver la transaccionalidad).

Descripción de la Práctica

Después de una nueva ronda de reuniones con el grupo inversor, están muy satisfechos del estilo arquitectónico basado en microservicios que hemos propuesto por **CultureInDaHouse** en la PEC1. Sin embargo, nos han pedido añadir nuevos requisitos al catálogo de ocio virtual:

- Con el fin de aumentar el *engagement* de los usuarios de la plataforma, cuando se cree un nuevo acto, nos piden que se envíe una notificación (correo electrónico) a todos los usuarios que tengan como favorita cualquiera de las categorías del acto. Por tanto, será necesario que los usuarios puedan crear y consultar sus categorías favoritas.
- Tecnológicamente, nos piden que la persistencia de la plataforma esté soportada por JPA y MySQL.

Ejercicio 1 (3 puntos)

En base a todos los requisitos establecidos en la especificación del caso **CultureInDaHouse**, especificar la arquitectura de microservicios de la aplicación correspondiente al caso:

1. Operaciones del sistema (1 punto). Usando el patrón CQRS, separar los **comandos** de las **consultas**:

Para cada uno de los **comandos** especificar:

- Actor
- Caso de uso
- Nombre del comando/*command*
- Descripción

Para cada una de las **consultas** especificar:

- Actor
- Caso de uso
- Nombre de la consulta/*query*
- Descripción

NOTA: Se debe usar una nomenclatura tipo “Lower Camel Case” (<https://wiki.c2.com/?LowerCamelCase>) para los nombres de los comandos y consultas.

2. Especificación de los comandos (1 punto). Para cada comando, especificar sus parámetros, el valor de retorno, las precondiciones y sus post-condiciones.

3. Modelo general de dominio (1 punto). Diagrama a alto nivel del modelo de dominio.

Solución

1. Operaciones del sistema

Los clientes web se comunicarán con HTTP, pero puede haber clientes que se comuniquen con mensajería (colas de mensajes). Para no estar ligados pues a un protocolo específico, hacemos primero esta abstracción en operaciones del sistema para representar las diferentes peticiones que puede recibir nuestro sistema.

Las operaciones del sistema las dividimos, usando el patrón CQRS, en:

- Comandos. Operaciones del sistema que crean, actualizan y borran datos
- Consultas. Operaciones del sistema que leen o consultan datos

*NOTA: La palabra acto en inglés es "event", pero se puede confundir con una "event architecture". Para referirnos a un acto cultural usaremos **Show** y a cada una de sus actuaciones **Performance**.*

Comandos (Commands):

Actor	Caso de uso	Comando	Descripción
Administrador	Dar de alta una categoría	createCategory()	Crear una categoría de actas
Administrador	Crear un acto	createShow()	Crear un acto sin ninguna actuación
Administrador	Crear una actuación	createPerformance()	Crear una actuación dentro de un acto
Administrador	Abrir un acto	openShow()	Hacer un acto visible para su compra. Se puede especificar la fecha de apertura para que lo haga el sistema automáticamente
Administrador	Cancelar un acto por completo	cancelShow()	Cancelar completamente un acto
Administrador	Cancelar una actuación específica	cancelPerformance()	Poner una actuación específica como cancelada.
Usuario no registrado	Registrarse al sistema	signUp()	Registrarse en el sistema.
Usuario	Modificar los datos	updateProfile()	Actualizar los datos

	personales		personales
Usuario	Reservar una actuación	createReservation()	Reservar una actuación dentro de un comando
Usuario	Finalizar comando	createOrder()	Finalizar la compra con las actuaciones reservadas
Usuario	Crear un comentario con puntuación sobre una actuación	createReview()	Crear un comentario sobre una actuación, añadiendo una puntuación entre 1 y 5
Usuario	Añadir una categoría a sus favoritos	addFavouriteCategory()	Añadir una categoría a la lista de categorías favoritas del usuario

Consultas (Queries):

Actor	Caso de uso	Query	Descripción
Usuario no registrado	Consultar las categorías disponibles	findCategories	
Usuario no registrado	Buscar un acto por nombre	findShowsLikeName	
Usuario no registrado	Buscar actas de una categoría	findShowsByCategory	
Usuario no registrado	Consultar la información de un acto	getShowDetails	Obtener información de un acto
Usuario no registrado	Consultar las actuaciones de un acto	getShowPerformances	Obtener las actuaciones previstas de un acto
Usuario	Consultar la lista de categorías favoritas	getFavouriteCategories	
Usuario	Ver el histórico de comandos realizados	findOrders	Encontrar todos los pedidos realizados
Usuario	Consultar las reservas de un	getOrderReservations	

	comando		
--	---------	--	--

*NOTA: No se representa ninguna operación del sistema relacionada con el login, la autenticación es gestionada con el componente **API Gateway**, por tanto no se considera una necesidad real del dominio y se considerará incorrecto las soluciones que hayan representado operaciones de login.*

2. Especificación de los comandos

- **createCategory**

Operación: createCategory(name, description)

Retorno: categoryId

Precondiciones: No existe previamente la categoría

Postcondiciones: La categoría es creada

- **createShow**

Operación: createShow(categoryId, name, description, image, price, capacity, duration)

Retorno: showId

Precondiciones: La categoría existe previamente

Postcondiciones: El acto se crea sin ninguna actuación, en el estado CREATED. El acto no está disponible todavía para su compra. Se notifica con correo electrónico la disponibilidad de este nuevo acto a todos los usuarios que tienen como la categoría del acto como una de sus categorías favoritas.

- **createPerformance**

Operación: createPerformance(showId, date, isPublic, capacity, price)

Retorno: performanceId

Precondiciones: El acto existe

Postcondiciones: Se crea una actuación con el estado CREATED (no se da todavía la opción de comprar tickets).

- **openShow**

Operación: openShow(showId, date=today)

Retorno: True/False

Precondiciones: El acto está en estado CREATED

Postcondiciones: El acto y todas sus actuaciones pasan a estado OPENED y el atributo onSaleDate contiene la fecha pasada (si no se pasa cabe fecha, se usa hoy como fecha).

- **cancelShow**

Operación: cancelShow(showId)

Retorno: True/False

Precondiciones: El acto está en estado CREATED o OPENED

Postcondiciones: El acto y todas sus actuaciones pasan a estado CANCELLED.

- **cancelPerformance**

Operación: cancelPerformance(performanceId)

Retorno: True/False

Precondiciones: La actuación está en estado OPENED

Postcondiciones: La actuación pasa en estado CANCELLED.

- **signUp**

Operación: signUp(email, password, fullName, mobileNumber)

Retorno: True/False

Precondiciones: -

Postcondiciones: El usuario queda registrado al sistema.

- **createReservation**

Operación: createReservation(userId, showId, performanceId, numberOfSeats)

Retorno: reservationId

Precondiciones: Existen el usuario y el acto

Postcondiciones: Se crea una reserva con la fecha de hoy.

- **createOrder**

Operación: createOrder(reservationIds)

Retorno: orderId

Precondiciones: Las reservas existen

Postcondiciones: Se autoriza el pago

- **createReview**

Operación: createReview(showId, comment, score)

Retorno: True/False

Precondiciones: El usuario no ha hecho previamente una revisión del mismo acto.

Postcondiciones: Se crea la revisión con la fecha del momento en el que se ha creado.

- **addFavouriteCategory**

Operación: addFavouriteCategory(userId, categoryId)

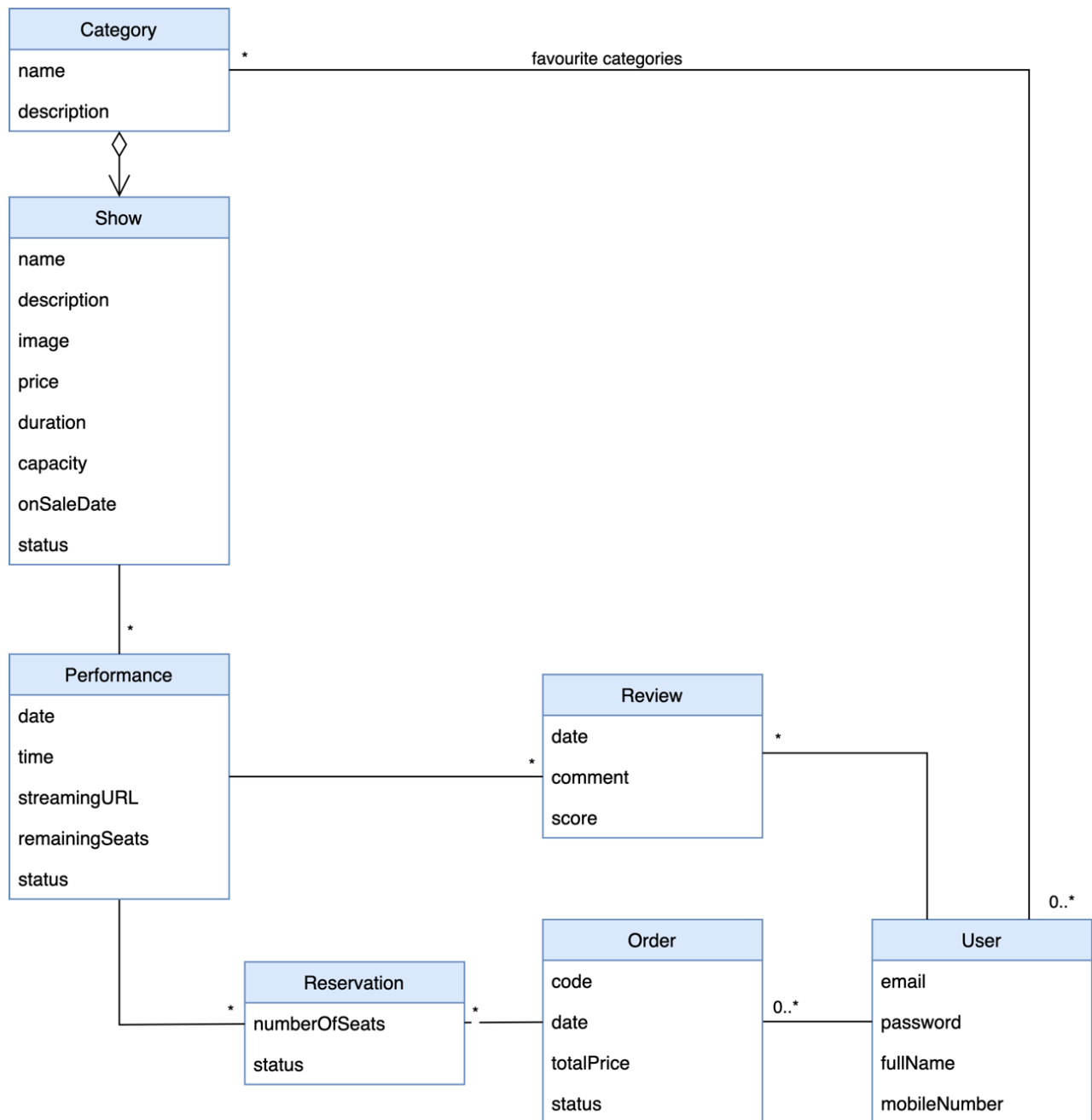
Retorno: True/False

Precondiciones: El usuario no tiene la categoría en la lista de favoritas.

Postcondiciones: Se añade la categoría a la lista de categorías favoritas del usuario.

3. Modelo de dominio (alto nivel)

El modelo de dominio a alto nivel describe las clases clave usando el vocabulario con el que describimos las operaciones del sistema.



Referencias:

- Microservices patterns: 2.2.1 "Creating a high-level domain modelo".
- Microservices patterns: 2.2.1 "Defining system operations".

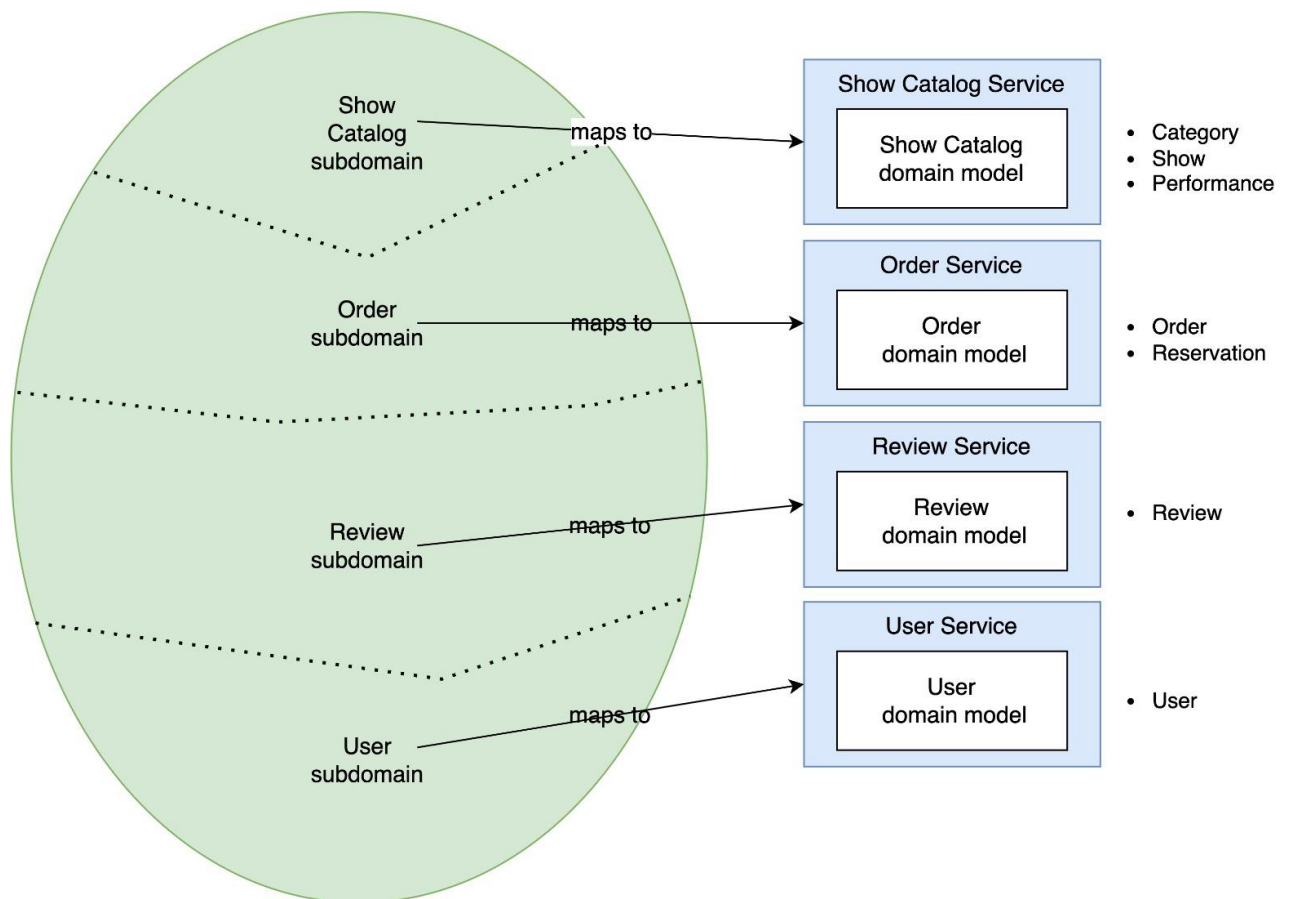
Ejercicio 2 (2 puntos)

En base a la definición de las operaciones del sistema y el modelo de dominio del ejercicio anterior, realizar la descomposición de los diferentes **microservicios** de la aplicación:

1. Identificar los subdominios y sus **servicios** necesarios, usando una descomposición basada en el patrón de sub-dominios (1 punto).
2. Realizar el **mapeo** entre las operaciones del sistema y los servicios identificados, e identificar las colaboraciones que requieren unos servicios de otros (1 punto).

Solución

1. Identificar los subdominios y lo suyos **servicios** necesarios, usando una descomposición basada en el patrón de sub-dominios.

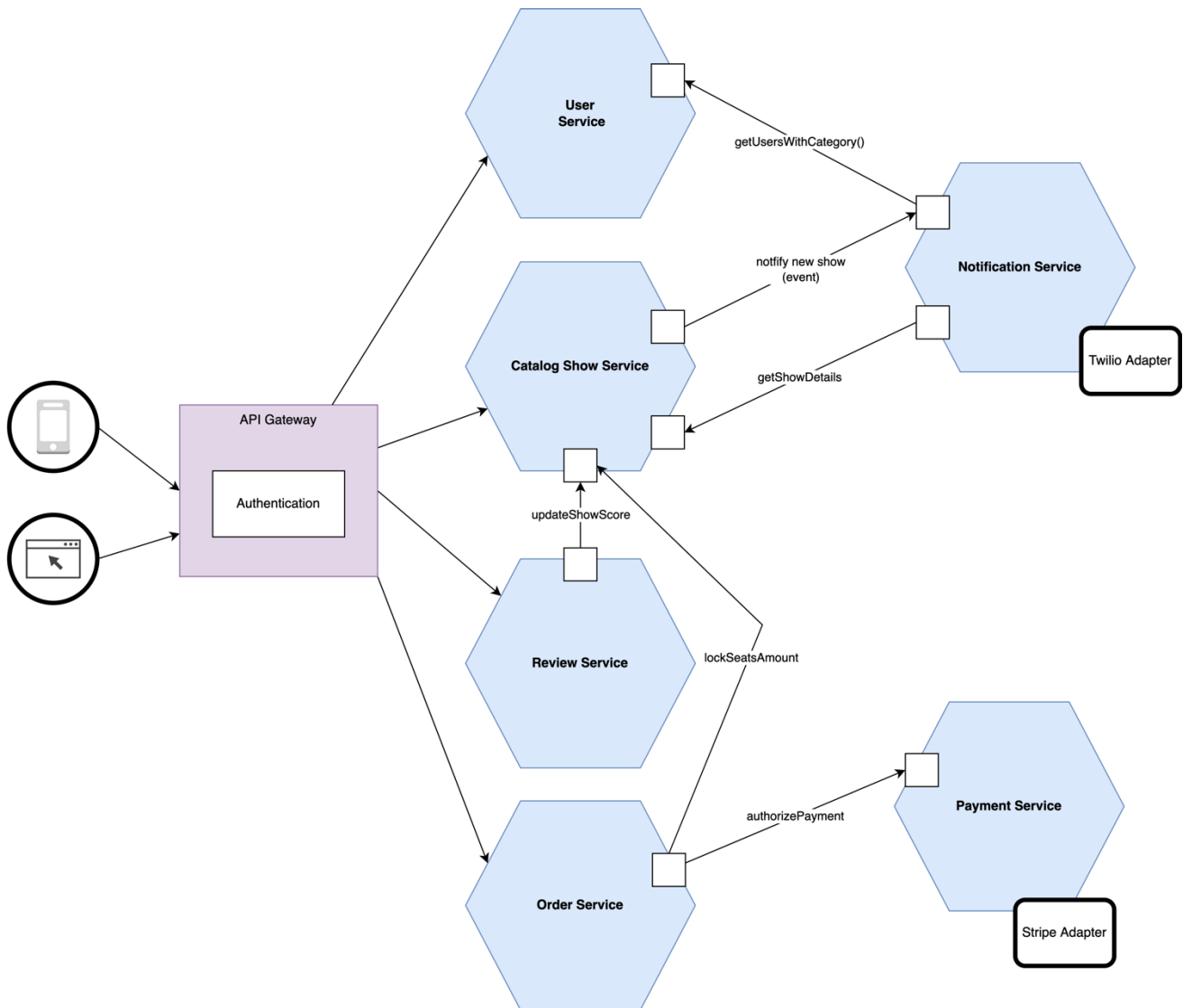


- **Show Catalog Service.** Gestionar el catálogo de categorías, actos y actuaciones disponibles.
- **Order Service.** Gestión de las reservas y pedidos realizados por los usuarios.

- **Review Service.** Gestionar comentarios y puntuaciones de los diferentes actos.
 - **User Service.** Gestionar el registro de usuarios.
 - **Payment Service.** Gestionar los pagos. No se ha definido ningún subdominio, puesto que no se quiere tener ninguna información del pago.
2. Para cada servicio identificado, establecer cuáles son las operaciones del sistema que resuelve, e identificar las colaboraciones que requieren otros servicios.

NOTA: En los pedidos que crean, actualizan y borran un modelo, por simplicidad solo se representa el pedido de creación.

Servicio	Operaciones	Colaboradores
Shows Catalog Service	▪ createCategory() ▪ createShow() ▪ createPerformance() ▪ openShow() ▪ cancelShow() ▪ cancelPerformance() ▪ findShowsLikeName() ▪ findShowsByCategory() ▪ getShowDetails() ▪ updateShowScore() ▪ lockSeatsAmount()	-
Order Service	▪ createReservation() ▪ createOrder()	▪ Shows Catalog Service: lockSeatsAmount() ▪ Payment Service: authorizePayment()
Review Service	▪ createReview() ▪ findShowReviews()	▪ Shows Catalog Service: updateShowScore()
Notification Service	▪ sendShowCreated()	▪ Shows Catalog Service: getShowDetails()
User Service	▪ signUp() ▪ addFavouriteCategory() ▪ getFavouriteCategories() ▪ updateProfile() ▪ getUsersByFavouriteCategory()	-
Payment Service	▪ authorizePayment()	-



Durante la descomposición podemos ver la necesidad de añadir operaciones internas que los microservicios tienen que ofrecer. Ejemplo: *lockSeatsAmount*, para que cuando se haga una reserva al *Show* se descuenten de *remainingSeats* el número de reservas realizadas por una actuación.

También hay decisiones que se pueden realizar para evitar un número innecesario de llamadas. Por ejemplo, el servicio **OrderService** podría tener una copia de todos los identificadores de los actos con el número de localidades disponibles.

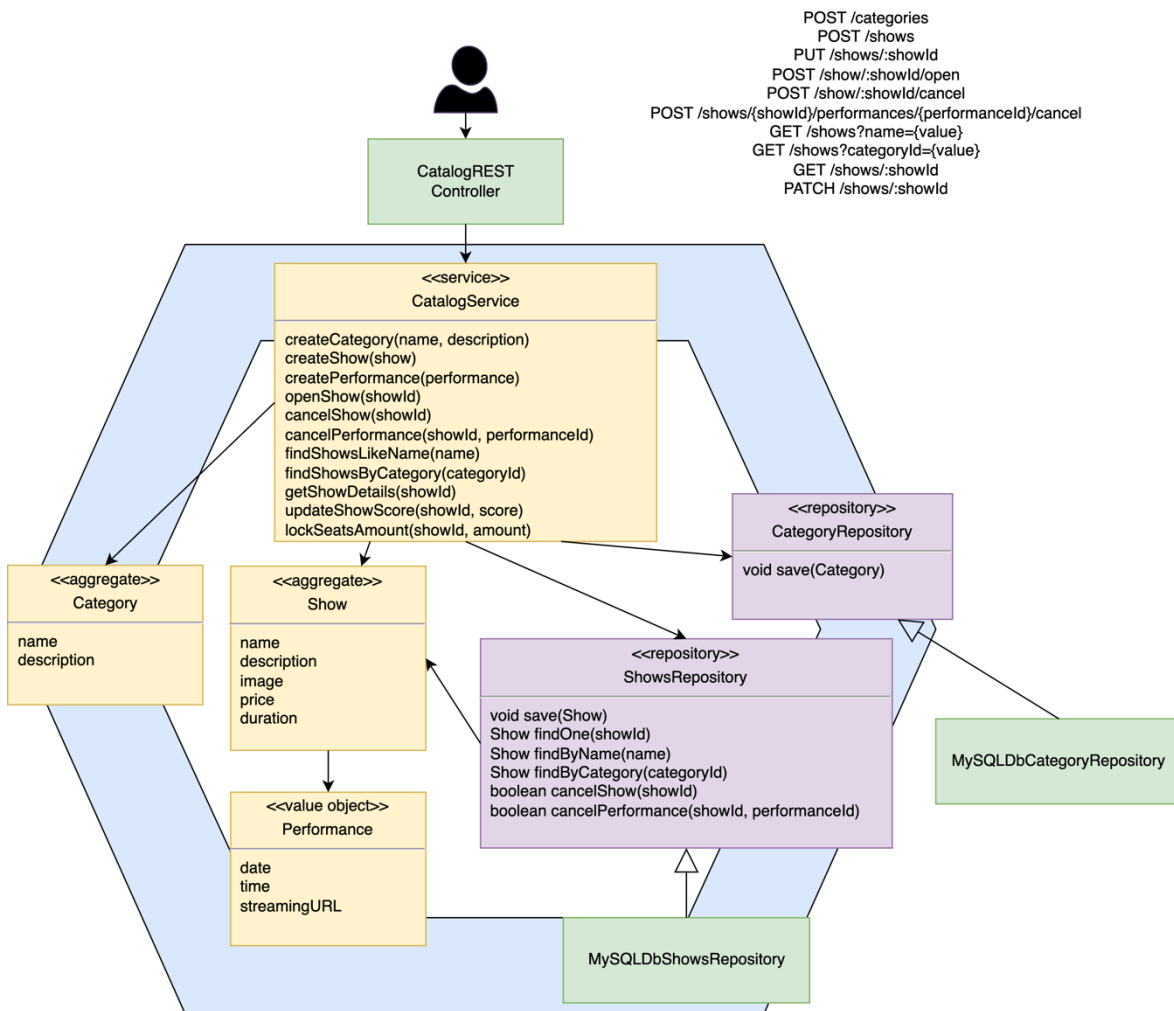
En el diagrama se muestran adaptadores de ejemplo reales que se podrían usar por los servicios de pago y de notificaciones.

Ejercicio 3 (3 puntos)

Partiendo de la descomposición realizada en el ejercicio anterior, detallar el **diseño** del servicio de catálogo de actos, teniendo en cuenta:

- El diseño se tiene que realizar usando una **arquitectura hexagonal**.
- El diagrama tiene que representar los **protocolos de comunicación** con el servicio (REST, etc.), y los **métodos y atributos** de las diferentes clases del servicio (no es necesario indicar los tipos).

Solución



Consideraciones:

- Toda actuación forma parte de un acto. Por este motivo el agregado es la actuación y todas las operaciones que se hacen sobre los actos se realizan desde el recurso actuación (*shows*).
- En relación con el punto anterior, para crear las actuaciones se hará uso del recurso PUT del acto.

Fuentes de información para la resolución del ejercicio:

- Microservices patterns - Capítulo 3, sección "Specifying REST APIs".
- Microservices patterns - Capítulo 5, sección "Designing a domain model using the DDD aggregate patterns".

Ejercicio 4 (2 puntos)

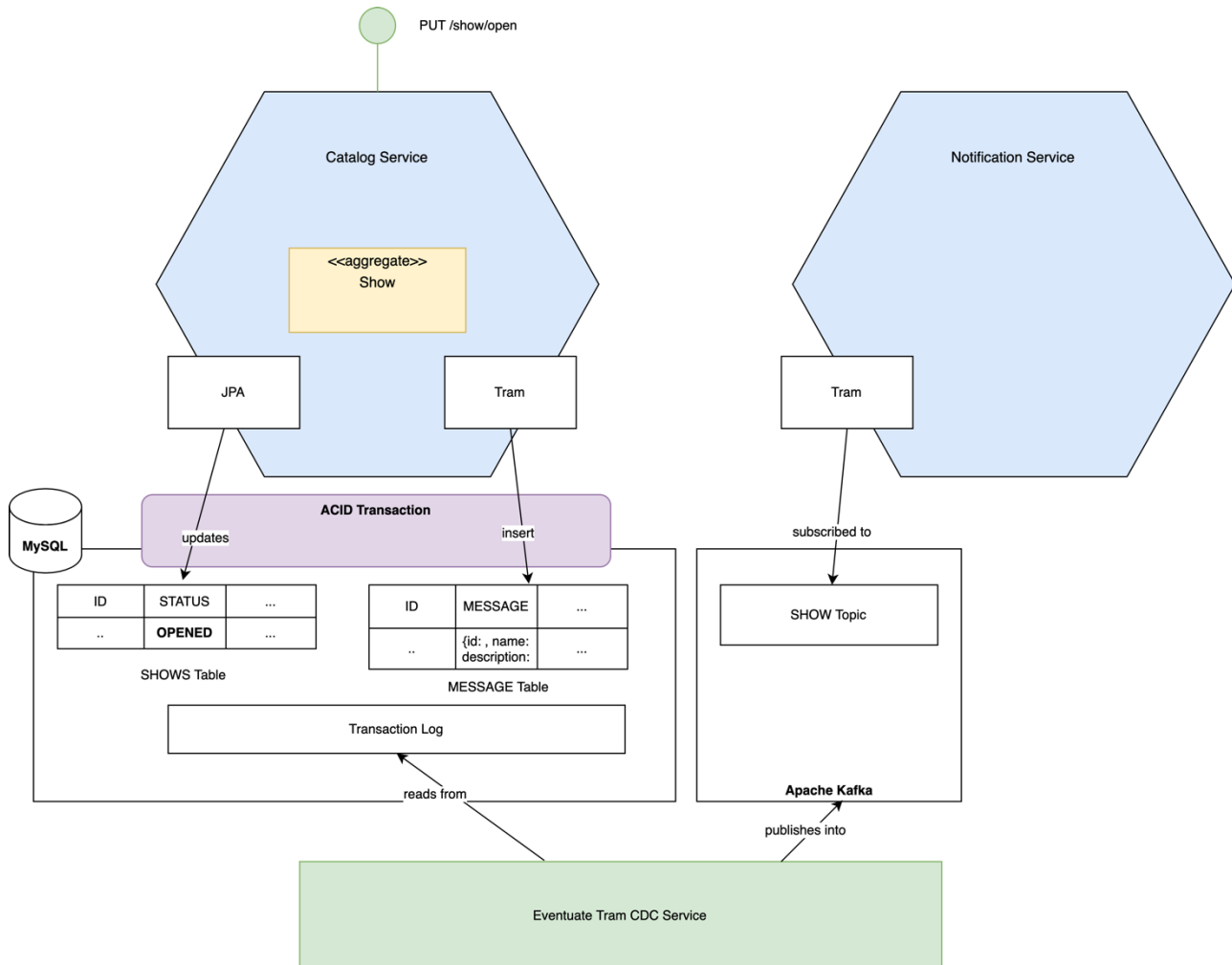
Cuando se realiza la apertura a la venta de un acto, queremos asegurar que se realiza de forma atómica:

- Actualización del **estado del acto** en la base de datos (a estado OPENED), y
- Envío del mensaje a una cola, indicando el **cambio de estado**. Este mensaje será consumido por el servicio de notificaciones, que realizará el envío de correos a todos los usuarios que estén suscritos a la categoría del acto.

Es decir, en caso de que por ejemplo falle la actualización de la base de datos del estado del acto, no se haría ningún envío del mensaje a la cola, y por tanto no se enviaría ningún correo de notificación a los usuarios.

Representar en un diagrama la solución a este planteamiento, usando el patrón o patrones de microservicios más adecuado/s. Justificar las decisiones tomadas.

Solución



El patrón escogido para resolver la problemática planteada es **Transactional Outbox**:
<https://microservices.io/patterns/data/transactional-outbox.html>

Fuentes de información para la resolución del ejercicio:

- Microservices patterns - Capítulo 3, sección "Communicating using the Asynchronous messaging".
- Microservices patterns - Capítulo 5.3, sección "Publishing domain events".

Recursos

Recursos Básicos

- Libro "Microservices patterns" (Richardson): Chapter 1. "Escaping monolithic hell".
- Libro "Microservices patterns" (Richardson): Chapter 2: "Decomposition strategies".
- Libro "Microservices patterns" (Richardson): Chapter 3: "Interprocess communication in a microservice architecture".
- Libro "Microservices patterns" (Richardson): Chapter 5: "Designing business logic in a microservice architecture".

Recursos Complementarios

- Sitio web: "A pattern language for microservices". <https://microservices.io/patterns/index.html>

Criterios de evaluación

- La Práctica 1 se tiene que resolver de forma **estrictamente individual**. En caso de detectar copias o cualquier otra forma de fraude académico, se penalizará la prueba con una D como nota.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Se debe entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PRAC1.pdf*.

Este documento se entregará en el espacio Registro de Evaluación Continua (REC) del aula antes de las **23:59 horas del día 8 de abril de 2022**. No se aceptarán entregas fuera de plazo.